

Antes del Lab — Tema 2: MAESTRO, Modelado de Amenazas Agénticas

Slides de apoyo técnico — se dan antes de correr el Lab 2.1

Qué vamos a necesitar entender

Los 3 labs de este tema convierten el framework MAESTRO (un ejercicio textual en la teoría) en herramientas que corren de verdad. Antes de leer el código hace falta tener claros 3 conceptos que todavía no cubrimos en detalle:

1. Qué es **MAESTRO** — el framework de 7 capas
2. Qué es "**structured output**" — necesario para entender el Lab 2.1
3. Qué es un **vector store** y **RAG** — necesario para entender el Lab 2.2

MAESTRO: 7 capas para modelar amenazas agénticas

MAESTRO descompone un sistema agéntico en 7 capas, de la más "profunda" (el modelo en sí) a la más "externa" (el ecosistema multi-agente):

7. Agent Ecosystem	(multi-agente, cross-agent)
6. Security & Compliance	(políticas, auditoría)
5. Evaluation & Observability	(logging, tracing)
4. Deployment Infrastructure	(contenedores, red)
3. Agent Frameworks	(ADK, orquestación)
2. Data Operations	(vector stores, RAG)
1. Foundation Models	(el LLM en sí)

Para cada amenaza, además, MAESTRO pide considerar 4 **factores agénticos** que no existen en el threat modeling tradicional: **non-determinism** (el LLM no responde siempre igual), **autonomy** (actúa sin supervisión constante), **identity management** (identidades de agentes) y

El arco de los 3 labs

Lab	Capa MAESTRO	Qué hace
2.1 — Generador de threat models	Las 7 capas	Un agente aplica el proceso MAESTRO completo a la descripción de un sistema y devuelve un threat model estructurado (JSON/Pydantic)
2.2 — RAG Poisoning	Capa 2 (Data Operations)	Un documento envenenado en un vector store se cuela como si fuera política oficial de la empresa
2.3 — Observabilidad	Capa 5 (Evaluation & Observability)	El mismo tipo de ataque (prompt injection → tool call → propagación) queda registrado con logging estructurado por capa, correlacionable entre capas

La progresión es: **modelar amenazas** → **ver una amenaza real** → **instrumentar para detectarla**.

Antes del Lab 2.1: ¿qué es "structured output"?

Normalmente un LLM responde con **texto libre** — párrafos, no datos.

"Structured output" es forzar al modelo a responder en una **forma exacta**, en este caso un JSON que coincide con un modelo de datos de **Pydantic** (una librería de Python para definir y validar esquemas):

```
class MaestroThreatModel(BaseModel):
    system_name: str
    layer_decomposition: Dict[str, LayerAnalysis] # 7 capas
    layer_threats: List[ThreatItem]
    cross_layer_threats: List[CrossLayerThreat]
    top_risks: List[ThreatItem]

agent = Agent(..., output_schema=MaestroThreatModel)
```

¿Por qué importa para un threat model automatizado? Porque el resultado no es para que un humano lo lea una vez — es para que **otro programa lo procese** (contar amenazas, filtrar por severidad, alimentar un dashboard). Texto libre no se puede parsear de forma confiable; un JSON con esquema fijo sí.

Ojo: que el JSON tenga la forma correcta **no garantiza que el contenido**

Antes del Lab 2.2: ¿qué es RAG?

El Lab 2.2 asume que entendés estos 3 términos:

- **Vector store:** una base de datos que guarda documentos como *embeddings* — vectores numéricos que capturan el significado del texto, no solo sus palabras exactas. Permite buscar por "similitud de sentido" en vez de coincidencia literal. En el Lab 2.2 es ChromaDB corriendo local.
- **RAG (Retrieval-Augmented Generation):** el patrón de "buscar antes de responder" — antes de que el LLM genere una respuesta, se recupera del vector store el/los documento(s) más relevantes para la pregunta y se los agrega al contexto, para que la respuesta esté fundamentada en esos documentos reales.
- **Provenance** ("procedencia"): de dónde viene un documento y si está verificado. En el Lab 2.2 cada documento tiene un campo de metadata `verified: True/False` — el parámetro `verified_only` de `search_knowledge_base()` decide si se descartan los documentos NO verificados antes de devolverlos al modelo. Esa es la mitigación central del lab.